Heart Rate Tracker — Backend API Spec

Mobile app sync & fetch contract for Chileaf HR device data (HR, RR, Sleep)

Overview

Mobile app Chileaf BLE heart rate device se data sync karta hai. Device internally last **48 hours** ka data buffer me rakhta hai (HR, RR intervals, Sleep). App is data ko fetch karke server pe upload karega aur baad me display ke liye fetch bhi karega.

Three data types:

• **HR** — heart rate readings (bpm), ~per-minute

• **RR** — R-R intervals (ms), ~per-heartbeat (high volume)

• **Sleep** — sleep stage sessions with stage array

Database Tables (4 tables)

```
-- 1. Upload metadata
CREATE TABLE device_sessions (
  id              BIGSERIAL PRIMARY KEY,
  athlete_id      UUID         NOT NULL,
  device_id       VARCHAR(64)  NOT NULL,
  device_name     VARCHAR(64),
  firmware_version VARCHAR(32),
  source          VARCHAR(32)  NOT NULL DEFAULT 'chileaf_hr',
  sync_from_ts    BIGINT       NOT NULL,
  sync_to_ts      BIGINT       NOT NULL,
  uploaded_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  hr_count        INT          DEFAULT 0,
  rr_count        INT          DEFAULT 0,
  sleep_count     INT          DEFAULT 0,
  INDEX idx_athlete_uploaded (athlete_id, uploaded_at DESC)
);

-- 2. Heart rate readings
CREATE TABLE hr_readings (
  athlete_id   UUID        NOT NULL,
  device_id    VARCHAR(64) NOT NULL,
  ts           BIGINT      NOT NULL,        -- epoch seconds (UTC)
  bpm          SMALLINT    NOT NULL,
  session_id   BIGINT      REFERENCES device_sessions(id),
  PRIMARY KEY (athlete_id, ts)
);

-- 3. RR intervals (high volume — consider monthly partitioning)
CREATE TABLE rr_intervals (
  athlete_id   UUID        NOT NULL,
  device_id    VARCHAR(64) NOT NULL,
  ts           BIGINT      NOT NULL,
  rr_ms        SMALLINT    NOT NULL,
  session_id   BIGINT      REFERENCES device_sessions(id),
  PRIMARY KEY (athlete_id, ts)
);

-- 4. Sleep segments (store as session array, NOT expanded per-minute)
CREATE TABLE sleep_segments (
  id            BIGSERIAL PRIMARY KEY,
  athlete_id    UUID         NOT NULL,
  device_id     VARCHAR(64)  NOT NULL,
  start_ts      BIGINT       NOT NULL,
  duration_sec  INT          NOT NULL,
  interval_sec  SMALLINT     NOT NULL,      -- usually 60
  stages        SMALLINT[]   NOT NULL,      -- [0,1,2,2,3,...]
  session_id    BIGINT       REFERENCES device_sessions(id),
  UNIQUE (athlete_id, start_ts)
```

```
);
```

Stage codes: 0 = awake, 1 = light, 2 = deep, 3 = rem

APIs (3 endpoints)

1. POST /api/v1/heart-tracker/sync

Upload 48hr data. Backend internally one transaction me 4 tables me bharta hai.

Request:

```
{
  "session": {
    "athleteId": "athlete_a1b2c3d4-e5f6-7890",
    "deviceId": "AA:BB:CC:DD:EE:FF",
    "deviceName": "Chileaf HR-Pro",
    "firmwareVersion": "1.2.3",
    "source": "chileaf_hr",
    "syncFromTs": 1747468800,
    "syncToTs": 1747641600,
    "uploadedAt": 1747641605
  },
  "hr": [
    { "ts": 1747468800, "bpm": 72 },
    { "ts": 1747468860, "bpm": 74 }
  ],
  "rr": [
    { "ts": 1747468800, "rrMs": 820 },
    { "ts": 1747468801, "rrMs": 815 }
  ],
  "sleep": [
    {
      "startTs": 1747555200,
      "intervalSec": 60,
      "stages": [0, 1, 1, 2, 2, 2, 1, 0]
    }
  ]
}
```

Response:

```
{
  "sessionId": 12345,
  "inserted":   { "hr": 2880, "rr": 170400, "sleep": 2 },
  "duplicates": { "hr": 0, "rr": 1500, "sleep": 0 },
  "status": "success"
}
```

Behavior:

• One DB transaction → 1 row in device_sessions, bulk insert in hr_readings / rr_intervals / sleep_segments

• Duplicate (athlete_id, ts) → silently skip (INSERT ... ON CONFLICT DO NOTHING)

• Accept Content-Encoding: gzip (RR data ~7 MB raw → ~800 KB gzipped)

2. GET /api/v1/heart-tracker/sync/last?athleteId=xxx

Last upload timestamp for delta sync.

Response:

```
{
  "lastSyncTs": 1747500000,
  "lastSessionId": 12344
}
```

App is value se newer data hi bhejega (bandwidth save).

3. GET /api/v1/heart-tracker/readings

Query params: athleteId, from, to (required); metric=hr,rr,sleep (optional filter); limit, offset (optional pagination, mainly for RR)

Example: /readings?athleteId=xxx&from=1747468800&to=1747641600

Response (same shape as upload — client model reuse):

```
{
  "athleteId": "athlete_a1b2c3d4",
  "deviceId": "AA:BB:CC:DD:EE:FF",
  "from": 1747468800,
  "to": 1747641600,
  "counts": {
    "hr": 2880,
    "rr": 170400,
    "sleep": 2
  },
  "hr": [
    { "ts": 1747468800, "bpm": 72 }
  ],
  "rr": [
    { "ts": 1747468800, "rrMs": 820 }
  ],
  "sleep": [
    {
      "startTs": 1747555200,
      "intervalSec": 60,
      "stages": [0, 1, 1, 2, 2]
    }
  ]
}
```

Conventions / Rules

Rule	Detail
Timestamps	Epoch seconds (UTC, 10-digit). Never milliseconds.
RR unit	rrMs is in milliseconds (typical range 300–2000)
Sleep stages	Array of int codes: 0=awake, 1=light, 2=deep, 3=rem
Empty arrays	Send [], do NOT omit the key
Sorting	All arrays sorted by ts / startTs ascending
Dedup	Client dedup before upload; backend dedup via ON CONFLICT
Compression	POST /sync supports Content-Encoding: gzip
Auth	Bearer token in Authorization header (existing pattern)

Volume Expectations (per athlete, per 48hr sync)

Metric	Approx rows	Notes
HR	2,000 – 3,000	per-minute readings
RR	150,000 – 200,000	per-beat — heaviest
Sleep	1 – 3 sessions	each with 300–500 element array

RR is the bottleneck — partition rr_intervals table by month and apply retention policy (e.g. keep raw 30 days, then aggregate).

Open Questions

- **Auth:** Bearer token use karein existing JWT se ya separate device-key?
- **Sleep interval:** Confirm SDK ka intervalSec value — client abhi 60s assume kar raha hai.
- **Retention:** RR raw data kitne din rakhna hai? (recommendation: 30 days raw + aggregated HRV)
- **Rate limiting:** Per-athlete kitne sync calls allow hain ek hour me?
- **Pagination:** GET response me default limit kya set karein?

Doc owner: Mobile team (Tushar) — confirm hone ke baad Dart side upload integration shuru hogi.